

CS7.401 Introduction to NLP – Spring 2026

Answer Key

End Semester Examination

Section A (10 marks)

- | | | | | |
|------|------|------|------|------|
| 1. C | 2. D | 3. B | 4. C | 5. A |
|------|------|------|------|------|

Section B (13 marks)

- | | | | | | |
|---------------|------|---------|------|---------|------------|
| 1. A, B, C, E | 2. C | 3. B, E | 4. C | 5. A, B | 6. A, D, E |
|---------------|------|---------|------|---------|------------|

Section C (5 marks)

- | | | | | |
|----------|----------|----------|----------|----------|
| 1. False | 2. False | 3. False | 4. False | 5. False |
|----------|----------|----------|----------|----------|

Section A, B, C are vetted by the faculty and will not be re-checked unless there was mistaken marking.

Section D

All answers must be grounded in reality. High-level "creative" assumptions that ignore data constraints or programmatic logic are not acceptable. Just having the keywords is not sufficient for grades.

1. Why do we need softmax and why can't we directly use logits from the model?
Explain with a small numerical example?
 - a. Why softmax 0.75 marks (with example)/ 0.4 without example
 - b. Why we can't use logit directly 0.75 marks (with example) | 0.4 without example
 - c. Example 0.5 marks

Softmax is used to convert logits (raw scores) into probabilities so that the outputs are interpretable and sum to 1.

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

For example, logits [2,1,0.1] become probabilities [0.659,0.242,0.099]

We cannot directly use logits because:

- they are not normalized (do not sum to 1),
- they cannot be used with cross-entropy loss.

Hence, softmax is required for classification tasks.

2. How is the root decided in a dependency tree?
 - a. The **main verb** is chosen as the root (1 mark). an example (0.5 Marks), AND If there is no verb, the most central word (e.g., a noun in titles or fragments - The end of the road) becomes the root (.5Mark). OR
 - b. In dependency parsing, the root is the main verb of the sentence and is defined linguistically. In transition-based parsing, the parser builds the tree using stack and buffer operations, and the root emerges as the final head of the sentence rather than being explicitly chosen to match transitions. (2 Marks)
3. Create a meaningful sentence in English without using a verb.
 - a. Answers have been verified with a PoS tagger in cases where there was a confusion and been marked in the sheet. AUX is auxiliary verb, is, am all forms of the verb 'to be'.
 - b. Clauses, Phrases are 0 unless they have ellipses.
 - c. Have marked if there is verb in sentences.
 - d. Partial marks if you have given more than one example and only one is correct.
4. Define what is finetuning and explain in brief how it is different from training. (1 Mark each – definition and difference)
 - a. **Fine-tuning** is the process of taking a **pre-trained model** and further training it on a **smaller, task-specific dataset** to adapt it to a new task.

b. Training learns all model parameters from scratch with random initialization, whereas fine-tuning continues learning from already learned weights, requiring less data and computation.

5. Why might we want to ensure that the input to the self-attention mechanism is the same size as the output? (1 Mark each)

- a. So that we can **stack multiple layers** on top of each other without changing the shape of the data (sequence length and vector size), while still refining the representations at each layer.
- b. It also enables **residual (skip) connections**, where the input is added to the output, which requires matching dimensions.

6. How can LoRA be explained by SVD conceptually?

- a. What it is,
 - i. LoRA can be explained using the idea behind **SVD**, where a matrix is approximated using a few important components (low-rank approximation). Similarly, LoRA assumes that the required weight update can be represented as a **low-rank matrix**, written as
$$\Delta W = AB \Delta W = AB$$
, instead of updating the full matrix.
 - ii. Thus, like SVD keeps only the most important directions, LoRA captures the main changes in a compact low-rank form, reducing the number of trainable parameters.
- b. What it isn't, (if any of following is present or insinuated then answer is not checked further and graded as 0).
 - i. LoRA reduces the size of model or is model compression.
 - ii. LoRA : SVD is used to decompose the weight matrix so that it can be trained or finetuned etc.

Section E:

All answers must be grounded in reality. High-level "creative" assumptions that ignore data constraints or programmatic logic are not acceptable. Just having the keywords is not sufficient for grades.

1. Any of the following is given 3 marks for contextual Variability while being in sync with OOV constraint (by sync we mean that these two together are programmable and consistent),

- a. ELMO – how you do get contextual embedding from forward and backward.
- b. Bilstm – how do you get contextual embedding from forward and backward.

Any of the following is given 3 marks for OOV (has to be mentioned, cant be that its implied, question was tell about the design, you have to look at how it can be achieved and is consistent with above constraint as well)

- c. Using Character CNN
- d. Any sort of sub word tokenization (BPE, subword, etc.)
- e. Masking

What is not given marks,

- Word2vec, glove: those by very definition are not dynamic embeddings. So no.
- Smoothing: This doesn't connect to question.
- you will get embedding of unknown word from context without mentioning BPE or any character level tokenization. (designs for both constraints have to be in sync!). If its said that we will get from context (without any mention of tokenization), How will you get embedding of W₁ in [W₍₋₁₎ W₍₀₎ W₍₁₎ W_(UNK) W₍₁₎], from surrounding context, yes. and W_(UNK) will come from? In this case its undetermined. Have to keep design consistent. If this kind of answer is given than marks are not given.

2. Bantu languages from Africa are agglutinative in nature and use prefixed noun declensions (eg ò-mú-límí, ò-mú-néné, ò-mú-kâddé). If you want BPE (Byte Pair Encoding) tokenizer to support these languages, what changes would you make? Support your answer with reasons.

1. Apply BPE: BPE can then learn subword units within stems without destroying prefix information. There is no difference. (this is mentioned with reasoning 6 Marks)

2. Limit merge operations / control vocabulary size: Prevents over-merging of tokens into full words and **Encourages** (doesn't mean ensures) retention of smaller meaningful units (in conjunction with step1. without adding additional little out of touch steps – check 4. 6 marks)

3. Train on a sufficiently large and representative corpus: Helps BPE learn frequent prefixes and morphemes as stable units (in conjunction with step 1. 6 marks)

4 Think programmatically how is that possible for African languages and all other low-resource languages. Checking all prefixes is not something that is possible. Most of them have data constraint issues. Solution has to be grounded in some reality. *Answers that are not grounded are awarded 0 marks.*

3.

- a. Using a Suffix Tree for subword tokenization (**1.5 marks**): A **suffix tree** stores all suffixes of training strings, allowing efficient discovery of **repeated substrings**.
 - i. Frequent substrings correspond to **useful subword units (morphemes)**
 - ii. Internal nodes represent **common prefixes of suffixes** → **repeated patterns**
 - iii. These patterns can be extracted as **candidate tokens**
- b. Supervised or Unsupervised? : Unsupervised. (**.5 marks**)
- c. Learning mechanism for optimal morphological split points (**2 marks**)
 - a. **Frequency-based assumption**
 - i. Substrings that appear frequently across words are likely meaningful units
 - b. **Branching in suffix tree**
 - i. A node with multiple children indicates **divergence point**
 - ii. These points suggest **possible split boundaries**
 - c. **Statistical criteria** (can be used):
 - i. High frequency of substring
 - ii. High branching factor
 - iii. Stability across contexts

OR

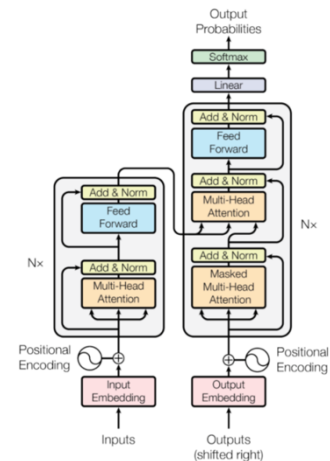
Optimal split points are identified using the suffix tree by selecting substrings that occur frequently and form internal nodes. Split boundaries are placed at points where branching occurs, indicating divergence into multiple continuations and thus meaningful morphological units.

- d. Tokenizing unseen input (1.25 marks for steps + .75 for example)
 - a. Start from the **root of suffix tree**
 - b. Traverse the tree matching the **longest possible substring**
 - c. When no further match is possible:
 - i. emit current substring as a token
 - ii. restart from root for remaining string
 - d. Repeat until full string is processed
- e. EXAMPLE For D.
 - a. Input String: aaabc
 - b. Start at root → match a → a → a → b (remaining c)
 - i. **Token 1:** aaab
 - c. Start at root → match c
 - i. **Token 2:** c

If BPE or morfeessor or sentence piece is used for tokenization, then what is the point of this question, these answers are not given marks.

Section F

1. Identify Word2Vec
 - a. Word2VecA – SkipGram Word2VecB – CBOW (1.5 marks)
 - b. Explanation on how CBOW and SkipGram work
 - i. In CBOW, the model predicts the current word based on the context of surrounding words. CBOW predicts the target word from its context. (1mark)
 - ii. Skip-Gram predicts context words given a target word. It's designed to learn the representation of a word by predicting the surrounding words in its context. (1mark)
 - c. Identifying the mean in the code as reason (1.5 marks)
 - d. -0.5 if anything explanation not clear or code line not clear
2. The correct order is - | `Class` a b c d | `def setup` e (h i g j) k l n o | `def call` f p u r s q m t
 - a. Marks not awarded for abcd
 - b. setup function – 5 marks - e (h i g j) k l n o
 - c. call function – 5 marks - f p u r s q m t
 - d. No other partial marks awarded
3. Attention mask matrix (10*10) for MLM – 2 marks, Attention mask matrix (10*10) for NWP – 2 marks. MLM in encoder – 0.5 marks; NWP in decoder – 0.5 marks. *(Partial marks – 1 awarded for incomplete but correct matrix)*
4. Fill the transformer boxes
 - a. 3 marks for getting boxes right
 - b. 2 marks for arrows.
 - c. Partial marks
 - i. 1 mark for correct encoder only
 - ii. 1 mark for correct decoder only
 - iii. None for getting just the arrows right



Section G

1. Morphological analysis and explain subunits (6 marks)
 - a. Ndiyabathanda → Ndi-ya-ba-thanda (2 marks)
 - b. **Ndi** (I) subject; **-ya-** tense; **-ba-** (them) object; **thanda** (love)verb root (4 marks)
 - c. Partial marks
 - i. each correct morpheme (0.5 marks)
 - ii. tense without correct morpheme (0.5 marks)
2. Tokenizer used and examples with tokenizer (3 marks)
 - a. Subword tokenier is used. (0.5 mark)
 - b. Why? – Reduces vocab size, Handles unseen, Captures recurring morphemes (3 * .5 marks) (Each correct reason is 0.5 marks)
 - c. Examples (1 mark) – should look like Ndiyabathanda = Ndi + ya + ba + Thanda (marks have been given even if the split is wrong for the tokenizer mentioned in a)
3. Architecture Overview and how model processes *Siyabathanda* → *We love them* (5 marks)
 - a. Input Handling – Tokenization + Embeddings + Positional Encoding (1 mark)
 - b. Encoder (if transformer) MHA for relationships and how it works. If other architecture, explanation should be clear and make sense (1 mark) (0.5 if no mention of how it works)
 - c. Decoder cross attention. How to get English output (1 mark) (0.5 if no output handling)
 - d. *Siyabathanda* = [Si, ya, ba, thanda] as input tokens / if other used it should make sense (1 mark)
 - e. Example decoder mapping ~ si → we; ba → them; Thanda → love and explain how the mapping works (1 mark)
4. *Niyabathanda* meaning, translation capacity, why (5 marks) - (*attention is wrong 0 marks*)
 - a. Meaning of *Niyabathanda* – *You love them* Meaning 1 mark; case + tense mentioned 1 mark
 - b. Model succeeds because of subword tokenization which allow for working with unknown tokens (1.5 mark) (0.5 marks for mentioning only tokenization)
 - c. Shared representation and vocabulary (1.5 marks)
5. Low resource setting (3 marks)
 - a. Finetuning over training from scratch for low resource languages (0.5 marks)
 - b. Doesn't need too much data, cheaper to train (0.5 marks)
 - c. Expected performance – Fair/good accuracy since it was trained on task (1 mark)
 - d. How to select model? Trained on multiple languages; trained on similar languages; (1 mark) (*If no explanation but name a model like mT5, mBART (0.5 marks)*)
6. Error analysis and fixes (3 marks) (*attention won't fix this 0 marks*)
 - a. Incorrect object translation - ba (them) confused with ndi (me); Limited data; morphological markers are subtle; model fails to identify object prefixes (0.5 marks if at least one of them is mentioned; 1 mark if more than one)
 - b. Architectural fix – Better subword segmentation / Incorporate morphological tagging and which component is responsible for that or how the change is made (1 mark)
 - c. Data fix – Add more training samples with various object markers and keep the data balanced (1 mark) (*If answer has only add more data – 0 marks*)